
PROJECT REPORT

BOOKSTORE

An Online Bookstore Web Application

Built with the MERN Stack (MongoDB, Express.js, React, Node.js)

<https://github.com/meghanacheppulla/BOOKSTORE>

Submitted in fulfilment of the full-stack web application project

Submitted By

BOOKSTORE Project Team

Report Date

July 2026

Table of Contents

1. Introduction	3
2. Key Features	3
3. Project Description	5
4. Scenario-Based Case Study	6
5. Technical Architecture	8
6. Database Design and Entity Relationships	10
7. Prerequisites	13
8. Setup and Installation Instructions	14
9. Project Structure	15
10. Application Flow	17
11. Frontend Configuration	18
12. Backend Configuration	19
13. Database Configuration	20
14. Environment Variables	21
15. Final Configuration and Running the Application	22
16. Verifying the Application	23
17. Additional Setup and Deployment	24
18. API Reference	25
19. Project Milestones	27
20. Application User Interface	29
21. Conclusion	33

1. Introduction

BOOKSTORE is a full-stack online bookstore web application engineered on the MERN technology stack, which combines MongoDB, Express.js, React, and Node.js into a single coherent platform. The application is intended to demonstrate an end-to-end e-commerce workflow in which customers can register accounts, browse a curated book catalogue, read and write reviews, build a shopping cart, and complete a simulated checkout. A separate administrative dashboard gives staff full control over the catalogue and the order pipeline, so the same codebase serves both shoppers and operators.

The project is deliberately structured as a self-contained minimum viable product. It implements the core capabilities required for a credible storefront, including authentication, role-based access control, full-text search, faceted filtering, cart persistence, order creation, and review management. Features that are commonly deferred in a learning or portfolio project, such as real payment gateway integration, transactional email, cloud image storage, and automated test coverage, are explicitly documented as Phase 2 enhancements rather than silently omitted. This clarity of scope makes the codebase a clean reference implementation for MERN architecture.

From a user-experience standpoint, BOOKSTORE is built around three personas. The first is the customer, whose journey runs from discovery through purchase and post-purchase review. The second is the reviewer, which is a role any authenticated customer can take on, because every logged-in user may submit one rating and comment per book. The third is the administrator, who manages the catalogue, monitors orders, and advances them through the fulfilment lifecycle. These personas map directly onto the application's role-based access control model, where a single User collection carries a role field whose value is either user or admin.

The remainder of this document describes the project in the depth required to understand, deploy, and extend it. It begins with the feature set and a narrative case study, then moves through the technical architecture, database design, prerequisites, installation, project structure, and application flow. Later sections cover configuration, the complete REST API reference, the milestone-based development plan, the live user interface, and a concluding summary of the engineering decisions that shape the platform.

2. Key Features

BOOKSTORE groups its capabilities into customer-facing features, administrative features, and cross-cutting features that apply to every user. The feature list below reflects the actual behaviour implemented in the repository and is the basis for the acceptance criteria of the application.

Customer Account Management

- Secure registration with email and password; passwords are hashed with bcrypt before storage and are never returned by any API response.
- Login with credential verification; a JSON Web Token is issued and persisted client-side for subsequent authenticated requests.
- Editable profile covering name, phone, and address; the email field is treated as an immutable identifier and disabled in the profile form.

- Automatic session restoration on page load through a bootstrap call to the current-user endpoint, which validates any stored token.

Book Browsing and Discovery

- A home page hero with a call to action and a Newest Arrivals strip that fetches the eight most recently added books.
- A search page offering keyword full-text search across title, author, description, and genre, debounced on the client for responsiveness.
- Faceted filtering by genre, price range, and minimum rating, with results sortable by newest, price ascending, price descending, or top rated.
- Pagination of search results and synchronisation of all filters into the URL query string so that views are shareable and bookmarkable.
- A book details page showing the cover, description, genre, rating summary, live stock indicator, quantity selector, and review section.

Shopping Cart

- Add to cart from either a book card or the book details page; quantity increments automatically if the book is already present.
- Inline quantity editing and item removal, with automatic removal when a quantity is reduced to zero.
- Persistence to browser local storage so the cart survives refresh, logout, and browser restart without server-side cart state.
- Live order summary showing the items total, shipping cost, and grand total, computed with the same rule enforced on the server.

Checkout and Order History

- A checkout page that pre-fills the shipping address from the user profile and offers three simulated payment methods: card, UPI, and cash on delivery.
- Order creation through a single endpoint that re-validates price and stock on the server, snapshots book titles and prices, and decrements stock.
- A profile page order history listing each order's short identifier, item count, total, status badge, and date for quick reference.

Reviews and Ratings

- Any authenticated user may submit one review per book, consisting of a one-to-five star rating and an optional comment of up to one thousand characters.
- A unique compound database index on book and user enforces the one-review-per-user-per-book rule at the data layer.
- Book average rating and rating count are recomputed automatically through Mongoose post-save and post-delete hooks whenever a review changes.

- Review authors may delete their own reviews, and administrators may delete any review, with the same recompute hook keeping ratings accurate.

Administrative Capabilities

- A dedicated admin dashboard accessible only to users whose role is admin, guarded both client-side and server-side.
- A books tab providing full create, read, update, and delete operations on the catalogue through an inline form and a confirmation-protected delete action.
- An orders tab listing every order with its customer, total, and status, plus an inline selector to advance status through pending, processing, shipped, and delivered, or to cancel.

Cross-Cutting Features

- Toast notifications for every user-facing action, rendered in an accessible live region and auto-dismissing after a short interval.
- Skeleton loading placeholders in the book grid while data is being fetched, giving immediate visual feedback before content arrives.
- Lazy-loaded route components for code splitting, so only the active page bundle is downloaded by the browser.
- A responsive layout using CSS grid with automatic fill and minimum column widths, collapsing to a single column with a toggled mobile menu below the seven hundred and sixty-eight pixel breakpoint.
- Security headers applied through Helmet, gzip compression for responses, a configurable cross-origin allow list, and a ten kilobyte body size limit to mitigate large-payload abuse.

3. Project Description

BOOKSTORE is a user-centric e-commerce platform that brings the conventional bookstore shopping experience to the web. The application connects readers with a catalogue of books through a streamlined digital interface, allowing users to search and filter titles, examine details, assemble a cart, and complete a purchase in a guided, multi-step flow. The design emphasises clarity and predictability, so that a first-time visitor can move from the landing page to a completed order without instructions.

For customers, the application offers secure registration and login, a searchable and filterable catalogue, a persistent cart, and a checkout flow that reuses the saved profile address. Each book carries a rating summary built from customer reviews, and every authenticated customer can contribute one review per book. The profile page consolidates personal details and order history in a single view, giving the customer a complete picture of their relationship with the store.

For administrators, the application provides a dashboard with two tabs. The books tab exposes the full catalogue management lifecycle, allowing new titles to be added and existing titles to be edited or removed. The orders tab presents every order placed in the system and lets the administrator advance each order through its fulfilment states. This separation of concerns ensures that catalogue management and order

management remain independent yet co-located, which simplifies day-to-day operations.

The application is built on a client-server architecture. The frontend is a single-page React application bundled with Vite, communicating with a RESTful Express.js backend over HTTP. The backend persists data in MongoDB through the Mongoose object document mapper, applies authentication and authorisation middleware, and enforces a consistent response envelope across all endpoints. Authentication is handled with JSON Web Tokens signed with a server secret, and passwords are protected with bcrypt hashing. This combination yields a platform that is straightforward to reason about, easy to extend, and representative of modern full-stack practice.

Because the project is scoped as a minimum viable product, several production concerns are intentionally deferred. Checkout is simulated, so no real payment is captured and orders are marked as paid immediately on creation. Stock decrements are not wrapped in a database transaction, which is acceptable for a single-instance deployment but would need to be revisited before scaling. There is no email service, so verification, password reset, and order confirmation messages are not sent. Each of these decisions is documented in the roadmap so that the platform can be hardened in well-defined stages.

4. Scenario-Based Case Study

The following case study traces a complete customer journey through BOOKSTORE, from first registration through order placement, review submission, and subsequent administrative action. Each scenario describes the user's intent, the interactions performed, and the system behaviour that results. Together they illustrate how the application's features combine into a coherent shopping experience.

Scenario 1: User Registration

A new visitor arrives at the BOOKSTORE landing page and decides to create an account. From the header navigation, the visitor opens the registration page and enters a full name, an email address, and a password of at least six characters. On submission, the client posts the credentials to the registration endpoint, where the server validates the input, hashes the password with bcrypt, and creates a User document with the default customer role. The server signs a JSON Web Token and returns it together with the safe user object. The client stores the token and cached user in local storage, displays a success toast, and redirects the visitor to the home page as an authenticated customer.

Scenario 2: Browsing the Catalogue

After logging in, the customer explores the catalogue. The home page greets the customer with a hero section and a Newest Arrivals strip showing the eight most recently added books. Each book card displays the cover image, title, author, genre badge, average rating, and price, with an add-to-cart button that is disabled and labelled Out of stock when the book has zero stock. The customer can click any card to open the book details page, or use the header search link to move to the full search interface.

Scenario 3: Searching and Filtering

On the search page, the customer types a keyword that is debounced and sent as a full-text query across the title, author, description, and genre fields. The customer then narrows the results by selecting a genre from the dropdown, setting a price range, and choosing a minimum rating threshold. Each filter change updates the URL query parameters and triggers a fresh paginated request. The customer sorts the results by top rated to surface the most acclaimed titles, and the grid updates with skeleton placeholders while the new results are fetched.

Scenario 4: Viewing Book Details

The customer selects a book and lands on its details page. The page shows the cover, title, author, genre, description, average rating with review count, and a stock indicator with a coloured dot. A quantity selector and an add-to-cart button let the customer place the book in the cart. Below the purchase controls, a reviews section lists existing reviews with the reviewer's name, rating, comment, and date, and provides a form for the customer to submit their own review if they have not already done so.

Scenario 5: Managing the Cart

The customer opens the cart page and finds the books added during the session, persisted across page refreshes by local storage. For each item, the customer can adjust the quantity inline or remove the item entirely. The order summary recalculates immediately, showing the items total, the shipping cost computed by the same rule as the server, and the grand total. When the cart total is above five hundred rupees, shipping is free; otherwise a flat forty rupee charge applies.

Scenario 6: Checkout and Order Placement

Satisfied with the cart, the customer proceeds to checkout. The checkout form is pre-filled with the shipping address from the profile. The customer selects a simulated payment method and submits the order. The server re-fetches each book to verify the current price and stock, snapshots the title and price into the order document, computes the totals, marks the order as paid, and decrements stock. On success, the client clears the cart, displays a confirmation toast, and navigates to the profile page, where the new order appears at the top of the order history.

Scenario 7: Writing a Review

After receiving and reading the book, the customer returns to the book details page to share an opinion. The customer selects a star rating, types a comment, and submits the review. The server creates a Review document linked to the book and the user, and the post-save hook recomputes the book's average rating and review count. The new review appears at the top of the review list, and the book's rating summary updates to reflect the additional feedback. If the customer later changes their mind, they can delete their review, which triggers the same recompute hook through the delete path.

Scenario 8: Administrative Catalogue Management

An administrator logs in with an admin-role account and opens the admin dashboard. On the books tab, the administrator reviews the full catalogue in a table with columns for title, author, genre, price, stock, and actions. The administrator adds a new book by opening the inline form, entering the required fields, and submitting; the book is created with the administrator recorded as the creator. Later, the administrator edits an existing book to restock it, and deletes an obsolete title after confirming the action in a dialogue.

Scenario 9: Administrative Order Management

On the orders tab, the administrator sees every order placed in the system, along with the customer's name and email, the total, and the current status. As orders are processed in the warehouse, the administrator advances each order from pending to processing, then to shipped, and finally to delivered, using an inline status selector. An order that cannot be fulfilled is marked as cancelled. These status changes are visible immediately to the customer in their order history, closing the loop between operations and the buyer.

Scenario 10: Platform Governance

Beyond day-to-day catalogue and order management, the administrator is responsible for the overall health of the platform. This includes reviewing the user base through the admin-only user listing endpoint, ensuring that public registration can only ever create customer accounts, and keeping the catalogue free of inappropriate content by removing books or reviews as needed. The combination of server-side role enforcement and ownership checks ensures that governance actions cannot be bypassed by manipulating the client.

5. Technical Architecture

BOOKSTORE follows a classic client-server architecture in which a single-page React application communicates with a stateless Express.js REST API, which in turn persists data to MongoDB. This separation of concerns allows the user interface and the business logic to evolve independently, and it mirrors the structure that most production MERN applications adopt. The request lifecycle is consistent across every endpoint: the client attaches a bearer token to each request, the server authenticates and authorises through middleware, the controller performs the business operation against the models, and a standard response envelope is returned to the client.

Frontend Technologies

The frontend is a React eighteen single-page application built and served in development by Vite. React Router version six handles client-side routing with a browser router, and routes are lazy-loaded through React.lazy and Suspense so that only the active page is fetched. Axios is the HTTP client; a configured instance attaches the bearer token from local storage to every request through a request interceptor and normalises error messages through a response interceptor. Styling is plain CSS backed by custom properties, with Google Fonts supplying Playfair Display for headings and Inter for body text, producing a warm, premium aesthetic without the overhead of a utility framework.

Backend Framework

The backend is an Express.js application running on Node.js. The server entry point wires together a middleware stack that begins with Helmet for secure headers, continues with compression for gzip responses, a ten kilobyte JSON body parser, a configurable CORS layer, and Morgan for request logging, and ends with the route mounts and a centralised error handler. The application is exported so it can be imported by a test harness without binding a port, and it only starts listening when executed directly. This pattern keeps the server easy to test and to embed.

Database and Authentication

MongoDB is the data store, accessed exclusively through the Mongoose object document mapper. Four models define the schema: User, Book, Review, and Order. Authentication is stateless and based on JSON Web Tokens. On registration or login, the server signs a token containing the user identifier, with a configurable expiry of seven days. Protected routes are guarded by a middleware that verifies the token, loads the user, and attaches it to the request. Passwords are hashed with bcrypt in a pre-save hook and are never selected by default, so they cannot leak through a careless query.

Admin Panel and Governance

Administrative capability is not implemented as a separate service. Instead, the User model carries a role field whose value is either user or admin, and a single authorisation middleware enforces role-based access control on the routes that require it. The admin dashboard is a protected route in the same single-page application, and every administrative mutation is re-checked on the server, so hiding a button in the interface is never the only protection. This approach minimises the attack surface while keeping the codebase unified.

Scalability and Performance

MongoDB scales horizontally and supports the indexing patterns the application relies on, including a text index on the Book model for full-text search and compound indexes for review uniqueness and order listing. The server applies gzip compression to reduce payload sizes, and the client lazy-loads route bundles to keep the initial download small. The response envelope includes pagination metadata for list endpoints, so large catalogues can be served efficiently. For higher traffic, the stateless API can be replicated behind a load balancer, and MongoDB can be deployed as a replica set to enable transactions, which the roadmap identifies as a prerequisite for transactional stock decrements.

Security Features

Security is layered. Helmet sets a baseline of secure HTTP headers. CORS is restricted to an allow list sourced from environment variables. The JSON body limit of ten kilobytes mitigates large-payload abuse. Passwords are hashed with bcrypt and never returned by the API. JSON Web Tokens are signed with a server secret and verified on every protected request. Role-based access control and ownership checks ensure that customers can only access their own orders and reviews, while administrators can access all of them. The combination of these measures provides defence in depth appropriate to a minimum viable product, with a clear roadmap for additional hardening such as refresh tokens and rate limiting before production scaling.

Notifications and User Feedback

Because the minimum viable product does not include an email service, user feedback is delivered entirely in the interface. Toast notifications confirm successful actions and report errors in an accessible live region, and skeleton loaders indicate pending data fetches. This in-band feedback keeps the user informed at every step without requiring out-of-band channels, and it leaves a clean insertion point for transactional email in a future phase.

6. Database Design and Entity Relationships

BOOKSTORE persists its data in MongoDB through four Mongoose models: User, Book, Review, and Order. The models are related by reference rather than by embedding, which keeps documents small and updates localised. The following subsections describe each entity, its fields and constraints, the indexes that support query performance and integrity, and the relationships that bind the entities into a coherent data graph.

User Entity

The User model represents every account in the system, whether customer or administrator. The name field is a trimmed string of two to sixty characters. The email field is unique, lowercased, trimmed, and validated against an email pattern; it serves as the immutable login identifier. The password field is a string of at least six characters, excluded from query results by default through the select-false option. The role field is an enum with values user and admin, defaulting to user. Optional fields capture a phone number and a shipping address composed of street, city, state, zip, and country. A pre-save hook hashes the password with bcrypt whenever it changes, and a safe-object method strips the password and version key before the document is returned to the client.

Field	Type	Constraints	Purpose
name	String	required, trim, 2-60 chars	Display name
email	String	required, unique, lowercase, email pattern	Login identifier
password	String	required, min 6, select false	Hashed credential
role	String	enum user/admin, default user	Access control
phone	String	optional, trim	Contact number
address	Subdocument	street, city, state, zip, country	Default shipping address
createdAt	Date	auto	Audit timestamp
updatedAt	Date	auto	Audit timestamp

Book Entity

The Book model represents a catalogue entry. The title, author, description, genre, and price fields are required, with length caps that prevent abuse. The stock field is a non-negative number that drives the add-to-cart button state and is decremented on order creation. The isbn field is unique but sparse, allowing multiple books without an ISBN. The coverImage field defaults to a placeholder. The publishedYear field is bounded between fifteen hundred and the current year. The ratingAverage and ratingCount fields are derived values maintained by Review hooks; ratingAverage is rounded to one decimal place and indexed in descending order to support top-rated sorting. A createdBy reference records the administrator who added the book. A text index spanning title, author, description, and genre powers full-text search, and single-field indexes on genre, price, and ratingAverage accelerate filtering and sorting.

Field	Type	Constraints	Purpose
title	String	required, trim, max 200	Book title
author	String	required, trim, max 120	Author name
description	String	required, max 3000	Marketing copy
genre	String	required, trim, indexed	Category for filtering
price	Number	required, min 0, indexed	Selling price in INR
stock	Number	required, min 0, default 0	Available units
isbn	String	unique, sparse	Optional identifier
coverImage	String	default placeholder	Cover image URL
publishedYear	Number	1450 to current year	Publication year
ratingAverage	Number	0-5, 1 decimal, indexed desc	Derived average rating
ratingCount	Number	default 0	Derived review count
createdBy	ObjectId ref User	optional	Admin who created the book

Review Entity

The Review model captures a customer's opinion of a book. The book and user fields are required references. The rating field is an integer from one to five. The comment field is optional, trimmed, and capped at one thousand characters. A unique compound index on book and user enforces the one-review-per-user-per-book rule at the data layer. A secondary index on book and creation date supports the newest-first listing. A static method recalculates the parent book's average rating and review count through an aggregation pipeline, and this method is invoked by post-save and post-find-one-and-delete hooks so that the derived fields stay in sync without any manual coordination.

Field	Type	Constraints	Purpose
book	ObjectId ref Book	required	Reviewed book

Field	Type	Constraints	Purpose
user	ObjectId ref User	required	Review author
rating	Number	required, 1-5	Star rating
comment	String	optional, trim, max 1000	Review text
createdAt	Date	auto	Audit timestamp
updatedAt	Date	auto	Audit timestamp

Order Entity

The Order model records a completed purchase. The user field references the customer. The items array contains subdocuments that snapshot the book reference, title, and price at order time, so historical orders remain accurate even if the book is later edited or deleted; the array must be non-empty. The shippingAddress subdocument captures the delivery details. The paymentMethod field is an enum of cash on delivery, simulated card, and simulated UPI. The itemsPrice, shippingPrice, and totalPrice fields are computed on the server and stored for audit. The status field tracks the fulfilment lifecycle through pending, processing, shipped, delivered, and cancelled. The isPaid and paidAt fields record the simulated payment. Indexes on user and creation date support the my-orders query, and an index on status supports administrative filtering.

Field	Type	Constraints	Purpose
user	ObjectId ref User	required	Ordering customer
items	Array of subdocuments	non-empty, no id	Line items with snapshot
items.book	ObjectId ref Book	required	Book reference
items.title	String	required	Title snapshot
items.price	Number	required	Price snapshot
items.quantity	Number	required, min 1	Units ordered
shippingAddress	Subdocument	street, city, state, zip, country	Delivery address
paymentMethod	String	enum, default card_simulated	Payment channel
itemsPrice	Number	required, min 0	Sum of line totals
shippingPrice	Number	required, min 0	Shipping cost
totalPrice	Number	required, min 0	Charged total
status	String	enum, default pending	Fulfilment state
isPaid	Boolean	default false	Payment flag
paidAt	Date	optional	Payment timestamp

Entity Relationships

The relationships among the entities are as follows. A User has many Orders, and each Order belongs to one User. A User has many Reviews, and each Review belongs to one User and one Book, with the unique compound index ensuring at most one review per user per book. A Book has many Reviews and may be referenced by many Order items; the Order item snapshots the title and price so that the order is insulated from later book edits. A Book also references the User who created it through the `createdBy` field. Administrators are not a separate collection; they are User documents whose role is admin, and this single mechanism is the entire basis for role-based access control in the data model.

7. Prerequisites

Before installing and running BOOKSTORE, the development environment must satisfy a small set of prerequisites that span the runtime, the database, and the foundational libraries. Each prerequisite is described below with its role in the project and guidance on obtaining it.

Node.js and npm

Node.js is the JavaScript runtime that executes the backend server and the frontend build tooling. The project requires Node.js version eighteen or later and has been verified against Node.js version twenty-two. The Node Package Manager, npm, is required to install dependencies and run the defined scripts; npm version nine or later is recommended. Node.js can be downloaded from the official website, and the installer includes npm. After installation, the `node` and `npm` commands should be available on the system path.

Express.js

Express.js is the web application framework for Node.js that underpins the backend. It provides routing, middleware composition, and HTTP request and response handling, making the backend scalable and maintainable. Express is installed as a project dependency through npm when the backend dependencies are installed, so no separate system-level installation is required.

MongoDB

MongoDB is the NoSQL database that stores user profiles, book catalogue entries, reviews, and orders. The project requires MongoDB version six or later. Developers may run a local instance of MongoDB on the default port, or they may use the free tier of MongoDB Atlas for a cloud-hosted database that requires no local installation. The connection string is supplied to the backend through an environment variable, so either option is supported without code changes.

React.js

React.js is the JavaScript library used to build the interactive, component-based user interface. The frontend is a React eighteen application, and React is installed as a project dependency through npm. No separate system-level installation of React is needed, because Vite scaffolds and bundles the application from the project dependencies.

Vite

Vite is the build tool and development server for the frontend. It provides fast hot-module replacement during development and an optimised production bundle. Vite is installed as a development dependency through npm, so no separate installation is required. The Vite configuration file enables the React plugin and configures the development server port.

Mongoose

Mongoose is the object document mapper that connects the Node.js backend to MongoDB and manages the schemas, validation, and hooks that govern the data. Mongoose is installed as a backend dependency through npm. Knowledge of Mongoose schemas, hooks, and population is helpful for understanding the data layer, and the official documentation provides a thorough reference.

HTML, CSS, and JavaScript

A working knowledge of HTML, CSS, and modern JavaScript is essential to understand and extend the project. The frontend is written in JSX, which is a syntax extension for JavaScript that couples markup with component logic, and the styling is plain CSS backed by custom properties. The backend is written in CommonJS JavaScript. Familiarity with asynchronous programming, the fetch and axios APIs, and the document object model will make the codebase easier to navigate.

8. Setup and Installation Instructions

BOOKSTORE is distributed as a two-package monorepo with a client directory for the frontend and a server directory for the backend. The following steps describe how to clone the repository, install dependencies, seed the database, and start both development servers. The steps assume the prerequisites listed in the previous section are already satisfied.

Clone the Project Repository

The project files are hosted on GitHub. Clone the repository to a local working directory using Git, then change into the project folder. The repository contains the client and server packages alongside a root README and a shared gitignore file.

```
git clone https://github.com/meghanacheppulla/BOOKSTORE.git
cd BOOKSTORE
```

Install Backend Dependencies

Change into the server directory, copy the example environment file to a real environment file, edit the environment variables to match the local MongoDB instance and a chosen secret, and install the backend dependencies. The installation pulls Express, Mongoose, jsonwebtoken, bcryptjs, express-validator, Helmet, CORS, compression, Morgan, dotenv, and the development tooling including nodemon.

```
cd server
cp .env.example .env
# edit .env: set MONGO_URI and JWT_SECRET
npm install
```

Seed the Database

Run the seed script to populate the database with an administrator account and six sample books. The script wipes the book, review, and order collections, idempotently creates the admin user so re-running it does not produce duplicates, and inserts the sample books. The seeded administrator credentials are taken from environment variables and default to the documented values.

```
npm run seed
```

Start the Backend Development Server

Start the backend in development mode with nodemon, which watches for file changes and restarts the server automatically. The API listens on port five thousand by default, and a health check endpoint is available to confirm that the server is running.

```
npm run dev
# API available at http://localhost:5000/api
```

Install Frontend Dependencies

In a second terminal, change into the client directory, copy the example environment file, and install the frontend dependencies. The installation pulls React, React Router, Axios, Vite, the Vite React plugin, and the ESLint tooling.

```
cd ../client
cp .env.example .env
npm install
```

Start the Frontend Development Server

Start the frontend in development mode with Vite. The dev server listens on port five thousand one hundred and seventy-three and proxies API requests to the backend through the configured base URL. Open the printed URL in a browser to use the application.

```
npm run dev
# UI available at http://localhost:5173
```

9. Project Structure

BOOKSTORE is organised as a two-package monorepo. The client directory holds the React frontend, the server directory holds the Express backend, and the repository root contains the README and the shared gitignore. The structure keeps frontend and backend concerns fully separated, which simplifies reasoning about the code and allows each package to be developed, built, and deployed independently. The full tree is presented below, followed by narrative descriptions of the frontend and backend parts.

Repository Tree

```
BOOKSTORE
|
+-- client/
|   +-- src/
|       |   +-- api/
|       |       |   +-- axios.js
|       |       +-- components/
|       |           |   +-- AdminBookForm.jsx
|       |           |   +-- BookCard.jsx
|       |           |   +-- BookList.jsx
|       |           |   +-- CartItem.jsx
|       |           |   +-- Header.jsx
|       |           |   +-- ProtectedRoute.jsx
|       |           |   +-- ReviewList.jsx
|       |           +-- context/
|       |               |   +-- AuthContext.jsx
|       |               |   +-- CartContext.jsx
|       |               |   +-- ToastContext.jsx
|       |           +-- pages/
|       |               |   +-- AdminDashboard.jsx
|       |               |   +-- BookDetails.jsx
|       |               |   +-- Cart.jsx
|       |               |   +-- Checkout.jsx
|       |               |   +-- Home.jsx
|       |               |   +-- Login.jsx
|       |               |   +-- Profile.jsx
|       |               |   +-- Register.jsx
|       |               |   +-- Search.jsx
|       |           +-- App.jsx
|       |           +-- index.css
|       |           +-- main.jsx
|       +-- index.html
|       +-- package-lock.json
|       +-- package.json
|       +-- vite.config.js
|       +-- .env.example
|       +-- .gitignore
|
+-- server/
|   +-- config/
|       |   +-- db.js
|   +-- controllers/
|       |   +-- authController.js
|       |   +-- bookController.js
|       |   +-- orderController.js
|       |   +-- reviewController.js
|       |   +-- userController.js
|   +-- middleware/
|       |   +-- auth.js
|       |   +-- errorHandler.js
|       |   +-- validate.js
|   +-- models/
|       |   +-- Book.js
|       |   +-- Order.js
|       |   +-- Review.js
|       |   +-- User.js
```

```
|   +-- routes/
|   |   +-- authRoutes.js
|   |   +-- bookRoutes.js
|   |   +-- orderRoutes.js
|   |   +-- reviewRoutes.js
|   |   +-- userRoutes.js
|   +-- seed/
|   |   +-- seed.js
|   +-- utils/
|   |   +-- apiResponse.js
|   +-- package-lock.json
|   +-- package.json
|   +-- server.js
|   +-- .env.example
|   +-- .gitignore
|
+-- README.md
+-- .gitignore
```

Frontend Part

The frontend is responsible for rendering the user interface and managing client-side state. The api directory contains the configured Axios instance with its interceptors. The context directory contains the three React context providers that hold authentication, cart, and toast state. The components directory holds the reusable building blocks, including the header, the route guard, the book card and list, the cart item, the review list, and the admin book form. The pages directory holds the top-level route components, one per application screen. The App.jsx file defines the routing table and the protected route wrappers, the index.css file holds all styling, and main.jsx bootstraps the React tree with the providers.

Backend Part

The backend handles server-side operations including authentication, data handling, and API responses. The config directory contains the database connection helper. The models directory defines the Mongoose schemas for User, Book, Review, and Order. The middleware directory contains the authentication and authorisation middleware, the validation result handler, and the centralised error handler. The controllers directory contains the business logic for authentication, books, orders, reviews, and users. The routes directory wires endpoints to controllers and applies validation chains. The seed directory contains the database seeding script. The utils directory contains the standard response envelope helper. The server.js file assembles the middleware stack and mounts the routes.

10. Application Flow

BOOKSTORE defines two primary user roles, customer and administrator, each with a distinct set of responsibilities and available actions. The application flow for each role is described below in terms of the sequence of actions a user performs and the system behaviour that results. These flows map directly onto the API endpoints and the route guards documented elsewhere in this report.

Customer Flow

- Create an account and log in using an email address and password, receiving a JSON Web Token that authenticates subsequent requests.
- Browse the home page to discover the newest arrivals, or open the search page to filter the catalogue by keyword, genre, price, and rating.
- Open a book's details page to read its description, check its stock and rating, and add it to the cart with a chosen quantity.
- Review the cart, adjust quantities, remove items, and observe the live totals before proceeding to checkout.
- Complete the checkout form with a shipping address and a simulated payment method, then place the order and receive a confirmation toast.
- Track the order in the profile page order history, where the status badge updates as the administrator advances the fulfilment lifecycle.
- Contribute a review to any book, with the rating and comment persisted and the book's average rating recomputed automatically.
- Cancel or delete a review previously submitted, which triggers the same recompute hook to keep the rating accurate.

Administrator Flow

- Log in with an administrator account whose role is admin, which is created by the seed script or by a direct database update.
- Open the admin dashboard, which is guarded by a protected route that requires the admin role both on the client and the server.
- On the books tab, add new books through an inline form, edit existing books to update details or stock, and delete obsolete titles after confirmation.
- On the orders tab, view every order placed in the system, filter by status, and advance each order through pending, processing, shipped, and delivered, or mark it cancelled.
- Govern the platform by reviewing the user base through the admin-only user listing endpoint and by removing inappropriate books or reviews as needed.
- Rely on server-side enforcement to ensure that administrative actions cannot be bypassed by manipulating the client interface.

11. Frontend Configuration

Configuring the frontend involves cloning the repository, installing dependencies, setting the API base URL, and starting the development server. The frontend is a Vite project, so the configuration is minimal and is driven by environment variables and the Vite configuration file. The following steps describe the process in detail.

Installation

Clone the repository from GitHub to a local machine, then navigate to the client directory where the React application is located. Install the dependencies with npm, which pulls React, React Router, Axios, Vite, and the supporting tooling. The installation creates a node_modules directory and a package-lock.json file that pins the resolved versions.

```
git clone https://github.com/meghanacheppulla/BOOKSTORE.git
cd BOOKSTORE/client
npm install
```

Environment Configuration

Copy the example environment file to a real environment file. The frontend requires a single variable that points to the backend API base URL. By default this is set to the local backend address. When deploying, this variable must be updated to the public URL of the deployed API so that the bundled frontend can reach it.

```
cp .env.example .env
# VITE_API_URL=http://localhost:5000/api
```

Start the Development Server

Start the Vite development server, which serves the application with hot module replacement for a fast development loop. The server prints a local URL, typically on port five thousand one hundred and seventy-three, that can be opened in a browser. Any change to the source files is reflected in the browser almost immediately.

```
npm run dev
# open http://localhost:5173
```

Production Build

To produce a production bundle, run the build script, which compiles the application into static assets in the dist directory. The preview script serves the built bundle locally for a smoke test. The dist directory can be deployed to any static host such as a content delivery network, an object store with a web frontend, or a conventional web server.

```
npm run build
npm run preview
```

12. Backend Configuration

Configuring the backend involves navigating to the server directory, installing dependencies, configuring the MongoDB connection, setting environment variables, and starting the server. The backend is a Node.js application using Express, so configuration is driven by environment variables loaded through dotenv.

Installation

Navigate to the server directory and install the backend dependencies with npm. The installation pulls Express, Mongoose, jsonwebtoken, bcryptjs, express-validator, express-async-handler, Helmet, CORS, compression, Morgan, and dotenv, along with the development dependency nodemon for automatic restarts during development.

```
cd BOOKSTORE/server
npm install
```

Configure MongoDB

Ensure that a MongoDB instance is available, either as a local service listening on the default port or as a cloud-hosted cluster on MongoDB Atlas. Open the environment file in the server directory and set the connection string to point to the chosen instance. For a local instance the string uses the localhost address; for Atlas it uses the cluster connection string with the database user credentials.

```
MONGO_URI=mongodb://127.0.0.1:27017/bookstore
# or for Atlas:
MONGO_URI=mongodb+srv://<user>:<password>@cluster0.xxxxx.mongodb.net/bookstore
```

Set Environment Variables

Create an environment file in the server directory to store environment-specific variables. Set the port, the JWT secret used to sign tokens, the token expiry, the allowed client origin for CORS, and the seeded administrator credentials. The JWT secret should be a long random string generated with a tool such as openssl. Replace placeholder values with strong secrets before any deployment.

```
PORT=5000
JWT_SECRET=<a long random string>
JWT_EXPIRES_IN=7d
CLIENT_ORIGIN=http://localhost:5173
SEED_ADMIN_EMAIL=admin@bookstore.com
SEED_ADMIN_PASSWORD=Admin@12345
```

Start the Backend Server

Start the backend server in development mode with nodemon, which watches for file changes and restarts automatically. In production, start the server with the start script and set the node environment to production. The server listens on the configured port and exposes the API under the api path, with a health check endpoint at the root of that path.

```
npm run dev      # development with auto-restart
npm start        # production
```

13. Database Configuration

BOOKSTORE uses MongoDB as its database. Developers can choose between a local installation and a cloud-hosted MongoDB Atlas cluster. Both options are fully supported because the connection is configured entirely through an environment variable. The following steps describe each option.

Local MongoDB Installation

Download and install MongoDB Community Server from the official MongoDB website for the operating system in use. After installation, start the MongoDB service, which by default listens on port twenty-seven thousand and seventeen. The connection string for a local instance points to the localhost address and the bookstore database name. Local installation is convenient for development and requires no internet connection once the binaries are installed.

```
MONGO_URI=mongodb://127.0.0.1:27017/bookstore
```

MongoDB Atlas Cloud Database

Create an account on MongoDB Atlas and provision a free-tier cluster. Define a database user with a strong password, allow network access from the development machine's IP address or from anywhere for development, and copy the connection string from the connect dialogue. Replace the placeholder in the connection string with the database user password and set it as the environment variable in the backend. Atlas removes the need for local installation and provides a managed, backed-up database.

```
MONGO_URI=mongodb+srv://<user>:<password>@cluster0.xxxxx.mongodb.net/bookstore
```

Verifying the Connection

After configuring the connection string, start the backend server and watch the console output. The database connection helper logs a success message when the connection is established and an error message if it fails. A quick way to confirm end-to-end connectivity is to run the seed script, which writes documents to the database and will report any connection error. The health check endpoint can also be requested to confirm that the server is running, though it does not itself test the database connection.

14. Environment Variables

BOOKSTORE is configured through environment variables loaded from gitignored environment files in each package. This keeps secrets out of version control and allows the same code to run in development and production with different settings. The tables below list every variable used by the backend and the frontend.

Backend Environment Variables

Variable	Description	Example
NODE_ENV	Environment name: development, production, or test	development
PORT	Port on which the API listens	5000
MONGO_URI	MongoDB connection string	mongodb://127.0.0.1:27017/bookstore
JWT_SECRET	Secret used to sign JSON Web Tokens	a long random hex string
JWT_EXPIRES_IN	Token lifetime	7d
CLIENT_ORIGIN	Allowed CORS origins, comma-separated	http://localhost:5173

Variable	Description	Example
SEED_ADMIN_EMAIL	Email of the admin created by the seed script	admin@bookstore.com
SEED_ADMIN_PASSWORD	Password of the admin created by the seed script	Admin@12345

Frontend Environment Variables

Variable	Description	Example
VITE_API_URL	Base URL of the backend API	http://localhost:5000/api

Environment files are gitignored at both the root and the per-package level, so real secrets are never committed. When deploying, the environment variables should be provided through the hosting platform's secret management facility rather than through a committed file. The JWT secret in particular must be a long, random value generated with a cryptographic source such as the openssl command, and it must be kept confidential because any party that possesses it can forge tokens.

15. Final Configuration and Running the Application

With the prerequisites installed, the repository cloned, the dependencies installed, the environment variables set, and the database seeded, the application is ready to run. The frontend and backend run as two separate processes, so two terminal sessions are required during development. This separation mirrors the production deployment model, where the frontend is served as static assets and the backend runs as a Node process.

Run Both Servers

Open two terminal windows. In the first, start the backend development server from the server directory. In the second, start the frontend development server from the client directory. The backend listens on port five thousand and the frontend on port five thousand one hundred and seventy-three. The frontend is configured to direct API requests to the backend through the environment variable, so no additional proxy configuration is required for local development.

```
# terminal 1 - backend
cd BOOKSTORE/server
npm run dev
```

```
# terminal 2 - frontend
cd BOOKSTORE/client
npm run dev
```

Running Both with a Single Command

For convenience, a tool such as concurrently can be installed as a development dependency to run both servers with a single command. Add scripts to the root or to a chosen package that invoke the backend and frontend start scripts in parallel. This is an optional convenience and is not required for the application to function.

```
npm install --save-dev concurrently
# then in package.json scripts:
"dev": "concurrently \"npm run dev --prefix server\" \"npm run dev --prefix client\""
```

Accessing the Application

After both servers are running, open the frontend URL in a browser. The application loads the home page, and the header navigation provides access to search, cart, login, and registration. Log in with the seeded administrator credentials to reach the admin dashboard, or register a new account to shop as a customer. The combination of the two running processes is the complete development environment.

16. Verifying the Application

After starting the application, a short verification pass confirms that both the frontend and the backend are functioning correctly. The verification steps below exercise the principal capabilities of the platform and surface any configuration issues early.

Check the Frontend

Open the browser at the frontend URL. The home page should render with the header navigation, the hero section, and the newest arrivals strip populated with book cards. Navigate to the search page and confirm that the filters and the book grid render. Open a book's details page and confirm that the description, rating, and add-to-cart control are visible. These checks confirm that the frontend is bundled correctly and that it can reach the backend.

Check the Backend

Use a browser or a command-line HTTP client to request the health check endpoint. The response should be a JSON object with a success flag, a message, and an uptime value. Then exercise the authentication flow by registering a user and logging in, which returns a token. Use the token to request the current-user endpoint, which confirms that token verification is working. Finally, request the books list endpoint to confirm that the seed data is reachable through the API.

```
curl http://localhost:5000/api/health

curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Test User","email":"test@example.com","password":"pass1234"}'
```

Check the Database

Connect to the MongoDB instance with a client such as mongosh or a graphical tool and inspect the bookstore database. The users, books, reviews, and orders collections should exist. The books collection should contain the six seeded sample books, and the users collection should contain the seeded administrator and any user registered during verification. This confirms that the connection string is correct and that the seed script ran successfully.

17. Additional Setup and Deployment

Beyond local development, BOOKSTORE can be placed under version control and deployed to cloud hosting platforms. The following subsections describe the optional additional setup steps that prepare the project for collaboration and for production.

Version Control

The repository is already initialised with Git and includes a gitignore file that excludes node_modules, environment files, build output, and logs. When extending the project, follow the branching convention of a main branch that is always deployable, a develop branch for integration, and feature branches named with a prefix such as feature, fix, or chore. Commit messages should follow the Conventional Commits specification, using prefixes such as feat, fix, docs, refactor, test, and chore so that the history is self-documenting.

Production Build

Build the frontend for production with the build script, which emits static assets to the dist directory. The backend requires no build step; it runs directly from source with the node environment set to production. Before building the frontend, set the API URL environment variable to the deployed backend's public address so that the bundled application targets the correct API.

```
cd client && npm run build
cd ../server && NODE_ENV=production npm start
```

Deployment Targets

Deploy the frontend static assets to any static hosting provider such as a content delivery network, an object store with web hosting, or a conventional web server. Deploy the backend to any Node.js hosting platform such as a platform-as-a-service provider, a container orchestration service, or a virtual machine. Point the deployed frontend at the deployed backend through the environment variable, and ensure that the backend's CORS configuration allows the deployed frontend origin. Use the hosting platform's secret management to provide environment variables rather than committing a real environment file.

Post-Deployment Verification

After deployment, repeat the verification steps from the previous section against the public URLs. Confirm that the health check endpoint responds, that registration and login succeed, that the catalogue loads, and that an order can be placed end to end. Rotate the JWT secret and the database credentials if there is any suspicion that they were exposed during development, and ensure that the production environment file is never committed to version control.

18. API Reference

The BOOKSTORE backend exposes a RESTful API under the `api` path. Every response uses a consistent envelope: a success flag, a message, and either a data field on success or an errors array on failure. Authentication is performed through a bearer token in the Authorization header for protected endpoints. The endpoints are grouped by resource into authentication, books, reviews, users, and orders, and each group is described below.

Response Envelope

```
// success
{ "success": true, "message": "...", "data": { ... } }

// error
{ "success": false, "message": "...", "errors": ["..."] }
```

Authentication Endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create an account; returns user and token; role forced to user
POST	/api/auth/login	Public	Authenticate credentials; returns user and token
GET	/api/auth/me	Private	Return the current user; used to bootstrap the session

Book Endpoints

Method	Endpoint	Access	Purpose
GET	/api/books	Public	Search, filter, sort, and paginate the catalogue; returns books and pagination metadata
GET	/api/books/genres/list	Public	Return the distinct list of genres for the filter dropdown
GET	/api/books/:id	Public	Return a single book by its identifier
POST	/api/books	Admin	Create a new book; sets the creator to the authenticated admin
PUT	/api/books/:id	Admin	Update an existing book with validation
DELETE	/api/books/:id	Admin	Delete a book from the catalogue
GET	/api/books/:bookId/reviews	Public	List reviews for a book, populated with reviewer names, newest first
POST	/api/books/:bookId/reviews	Private	Add a review; one per user per book; triggers rating recompute

Review Endpoints

Method	Endpoint	Access	Purpose
DELETE	/api/reviews/:id	Owner or Admin	Delete a review; triggers book rating recompute via the delete hook

User Endpoints

Method	Endpoint	Access	Purpose
GET	/api/users/profile	Private	Return the authenticated user profile as a safe object
PUT	/api/users/profile	Private	Update name, phone, and address; email and role are not editable
GET	/api/users	Admin	List all users sorted by creation date

Order Endpoints

Method	Endpoint	Access	Purpose
POST	/api/orders	Private	Place an order; server re-validates price and stock; snapshots title and price; decrements stock
GET	/api/orders/my	Private	List the authenticated user orders, newest first
GET	/api/orders/:id	Owner or Admin	Return a single order; forbidden if not the owner and not an admin
GET	/api/orders	Admin	List all orders; optional status filter; paginated
PUT	/api/orders/:id/status	Admin	Update the order status through the fulfilment lifecycle

Query Parameters for Book Search

The book list endpoint accepts a rich set of query parameters that drive the search page. The keyword parameter triggers a full-text search across the title, author, description, and genre fields. The genre, author, minPrice, maxPrice, and minRating parameters apply faceted filters. The sort parameter selects the ordering: newest, price ascending, price descending, or top rated. The page and limit parameters control pagination, and the response includes a pagination object with the total count, current page, total pages, and page size.

Parameter	Type	Description
keyword	String	Full-text search across title, author, description, genre
genre	String	Exact genre filter
author	String	Author name filter

Parameter	Type	Description
minPrice	Number	Minimum price inclusive
maxPrice	Number	Maximum price inclusive
minRating	Number	Minimum average rating inclusive
sort	String	newest, priceAsc, priceDesc, or ratingDesc
page	Number	One-based page number
limit	Number	Page size

19. Project Milestones

The development of BOOKSTORE can be decomposed into five milestones that together deliver the complete platform. Each milestone produces a verifiable increment and builds on the previous one. The milestones are presented in the order in which they would typically be executed, and each is described in terms of its goal, the work performed, and the outcome that confirms its completion.

Milestone 1: Project Setup and Configuration

Setting up a structured environment is the foundation of any application. By organising the project into separate folders for the frontend and the backend, the codebase becomes manageable and scalable from the outset. The frontend folder contains all code related to the user interface, written in JavaScript with React and Vite, and the backend folder manages the server, the API routes, and the database interactions through Node.js and Express. This separation maintains a clear boundary between UI logic and server logic and allows changes in one area without affecting the other.

In this milestone, the repository is initialised, the two packages are scaffolded, the shared gitignore is added, the example environment files are created, and the base dependencies are installed in each package. The frontend is configured to point at the backend through an environment variable, and the backend is configured to allow the frontend origin through CORS. The outcome is a runnable skeleton in which the frontend loads and the backend responds to the health check, even though no business logic exists yet.

Milestone 2: Backend Development

The backend forms the core logic and data management of the project. A well-structured backend ensures efficient data handling, security, and scalability. The Express server is assembled with a middleware stack that includes Helmet, compression, a JSON body parser, CORS, Morgan, the route mounts, and a centralised error handler. Routes are organised by resource into authentication, books, reviews, users, and orders files, each with a single responsibility, and each route is wired to a controller that performs the business operation.

Authentication is implemented with JSON Web Tokens signed on registration and login and verified by a protect middleware on every protected route. Authorisation is layered on top through an authorize middleware that checks the user's role. Passwords are hashed with bcrypt in a pre-save hook. A consistent response envelope is enforced through a utility helper so that every endpoint speaks the same shape. The

outcome is a complete, secure, and documented API that the frontend can consume.

Milestone 3: Database Development

Using MongoDB as the database provides a flexible, document-oriented structure that is well suited to the varied data of an e-commerce application. In this milestone, the four Mongoose models are defined with their fields, types, constraints, and indexes. The User model carries the role field that underpins access control. The Book model carries the text index that powers full-text search and the derived rating fields maintained by Review hooks. The Review model carries the unique compound index that enforces one review per user per book. The Order model carries the item snapshot subdocuments that insulate historical orders from later book edits.

Cross-cutting data behaviour is implemented through Mongoose hooks. The Review post-save and post-find-one-and-delete hooks call a static method that recomputes the parent book's average rating and review count through an aggregation pipeline, keeping the derived fields in sync without manual coordination. The seed script is written to populate the database with an administrator and sample books, and it is made idempotent for the administrator so it can be re-run safely. The outcome is a data layer that is consistent, indexed, and seeded with realistic sample data.

Milestone 4: Frontend Development

Frontend development focuses on creating an interactive, intuitive user experience through a React-based single-page application. The initial application structure is set up with Vite, and files are organised into components, context, pages, and api directories so that the project is easy to navigate and maintain. Reusable components such as the header, the book card, the cart item, and the review list are designed once and reused across pages, reducing development time and ensuring consistency.

Three React context providers hold the global state: authentication, cart, and toast notifications. Routing is handled by React Router with lazy-loaded pages and a protected route wrapper that mirrors the server-side role checks. Styling is plain CSS backed by custom properties, producing a warm, premium aesthetic with a single responsive breakpoint. Accessibility is baked in through semantic HTML, labelled controls, and an accessible live region for toasts. The outcome is a polished, responsive, and accessible interface that exercises every backend capability.

Milestone 5: Integration and Testing

The final milestone integrates the frontend and the backend and verifies the complete user journeys end to end. The frontend is pointed at the backend through the environment variable, and the principal flows are exercised: registration, login, browsing, search, cart management, checkout, order history, review submission, and administrative catalogue and order management. The centralised error handler is confirmed to convert Mongoose errors into meaningful HTTP responses, and the response envelope is confirmed to be consistent across every endpoint.

Although the minimum viable product does not include an automated test suite, the integration milestone establishes the cURL smoke tests that exercise the API and the manual verification checklist that exercises the interface. The roadmap captures the follow-on work of adding automated tests with a backend test

harness and a component testing library. The outcome is a verified, deployable platform that meets the documented acceptance criteria and is ready for the Phase 2 enhancements.

20. Application User Interface

The BOOKSTORE user interface is a responsive single-page application with a warm, premium aesthetic. A header with the brand, primary navigation, and a cart badge is present on every page. The principal screens are presented below with descriptions of their layout and the interactions they support. The screenshots illustrate the running application as seen by an authenticated customer.

Home Page

The home page greets the visitor with a hero section whose headline invites the reader to find their next great read, paired with a prominent call-to-action button that jumps to the full catalogue. Below the hero, a Newest Arrivals strip surfaces the eight most recently added books as cards, each showing the cover, title, author, genre badge, average rating, and price, with an add-to-cart control that is disabled and labelled when the book is out of stock. The page establishes the brand identity and gives the visitor an immediate sense of the catalogue.

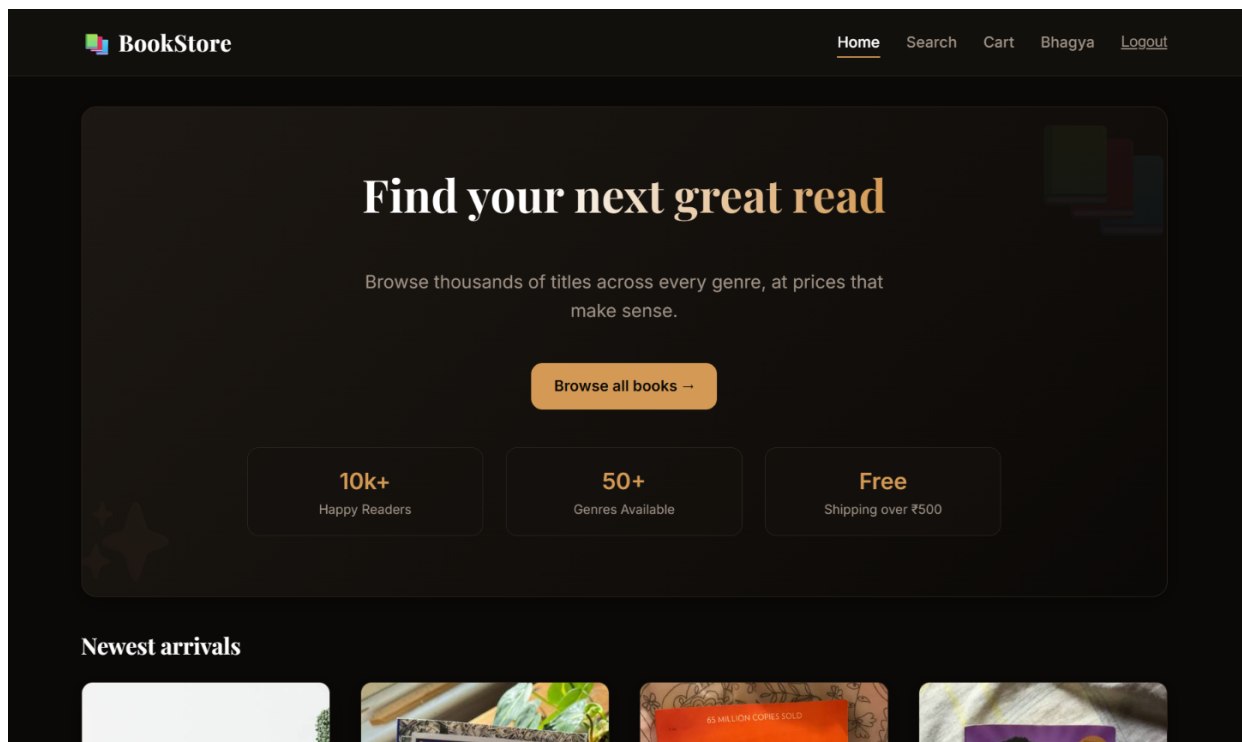


Figure 1. Home page hero section with the primary call to action.

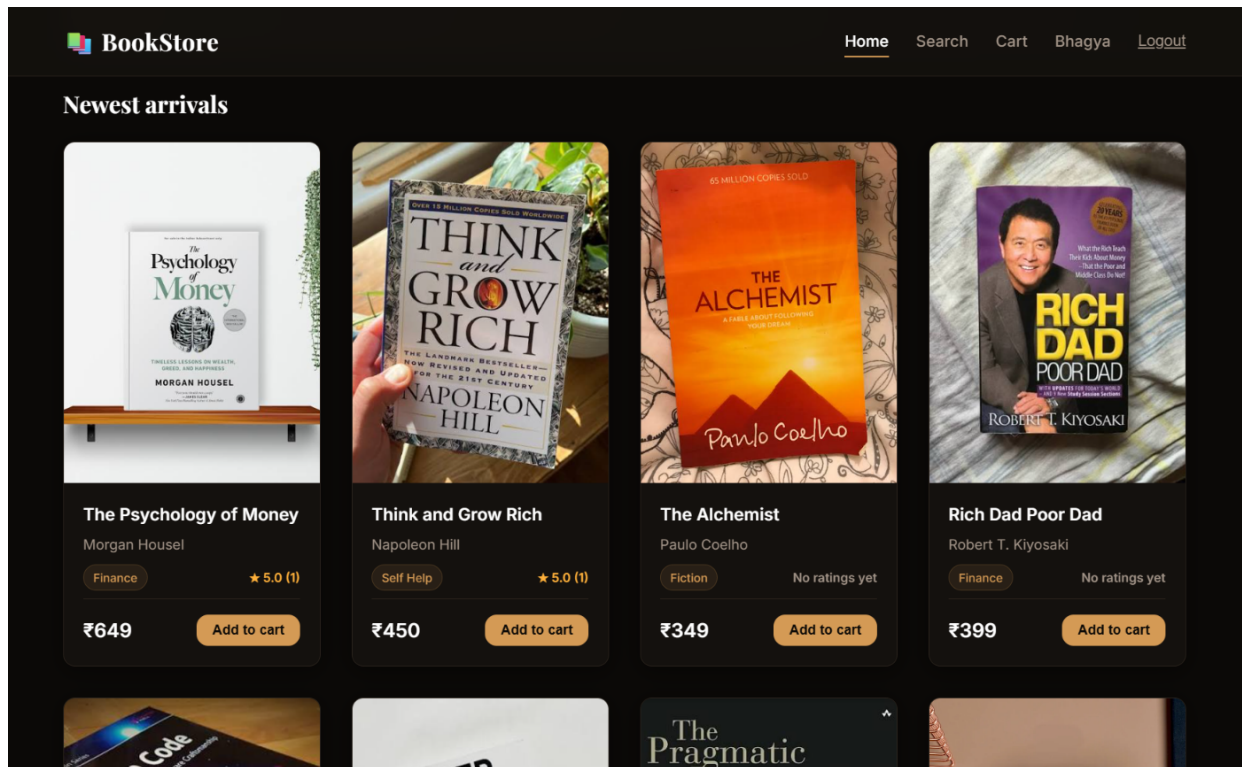


Figure 2. Home page newest arrivals strip showing book cards with ratings and add-to-cart controls.

Search Page

The search page is the discovery hub of the application. A keyword input is debounced and issued as a full-text query, and a sidebar of filters allows the customer to narrow the results by genre, price range, and minimum rating, with a sort selector for newest, price, and rating orderings. The results grid renders book cards with skeleton placeholders while data is fetched, and pagination controls appear when the result set spans multiple pages. Every filter is reflected in the URL query string, so views are shareable and bookmarkable.

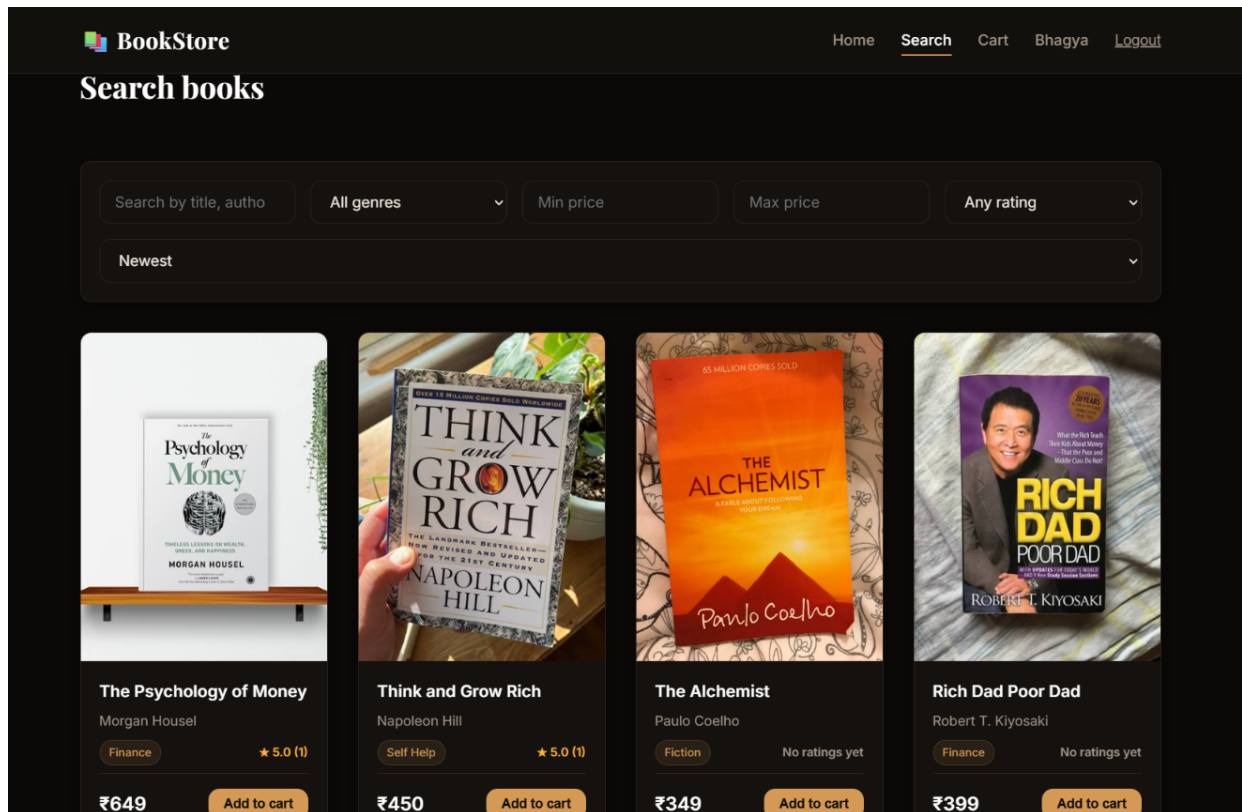


Figure 3. Search page with keyword input, faceted filters, sort selector, and results grid.

Cart Page

The cart page presents the items the customer has added, persisted across sessions by local storage. Each row shows the book's cover, title, and price, with a quantity input and a remove button. The order summary panel lists the items total, the shipping cost computed by the same rule as the server, and the grand total. A prominent call to action initiates the checkout flow. When the cart is empty, an empty-state message guides the customer back to the catalogue.

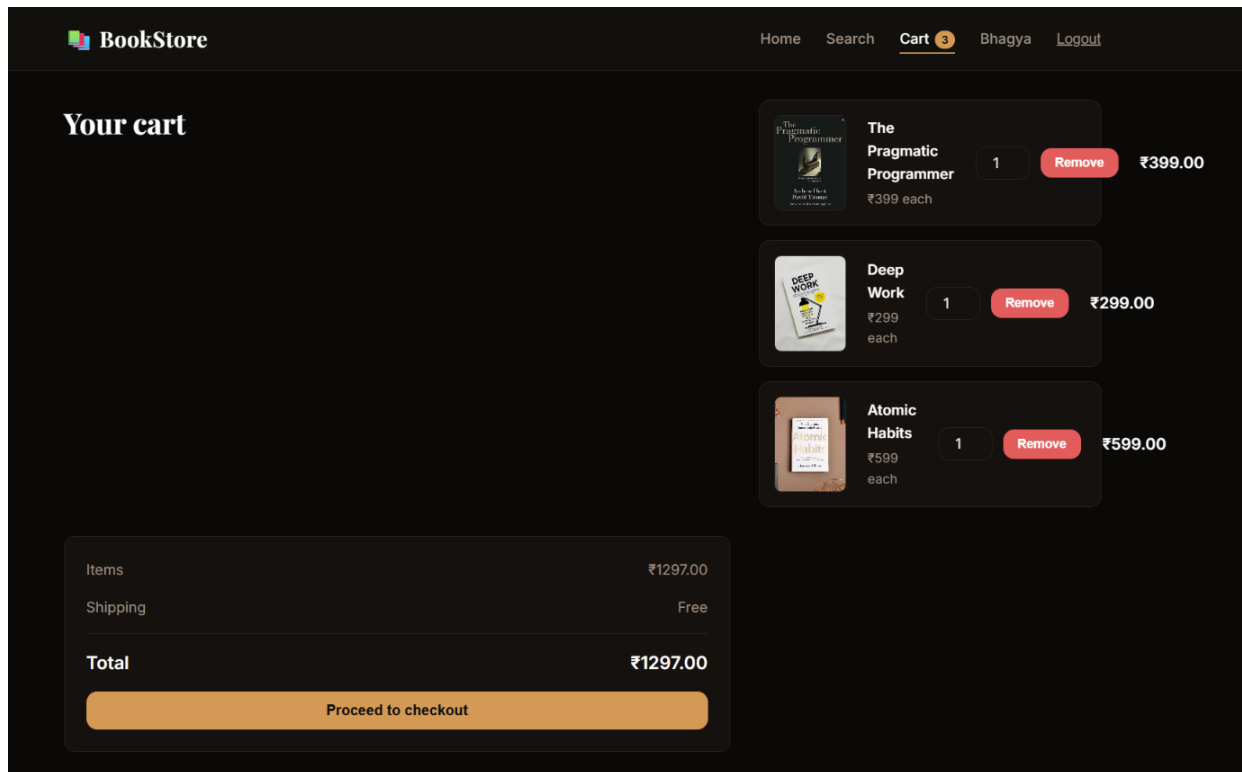


Figure 4. Cart page with line items, quantity controls, and the order summary including shipping and grand total.

Profile Page

The profile page consolidates the customer's personal details and order history in a single view. The upper section is an editable form for name, phone, and address, with the email field disabled because it is the immutable login identifier. The lower section lists the customer's orders, each with a short identifier, the item count, the total, a status badge, and the date, so the customer can track the fulfilment lifecycle at a glance. The page is the customer's home base after purchase.

BookStore Home Search Cart 3 Bhagya Logout

My profile

Name
Bhagya Prasad Dannina

Email
bhagyaprasaddannina92@gmail.com

Phone
6302535979

Street
peddapuram

City
peddapuram

State
Andhra Pradesh

ZIP / PIN code
533437

Figure 5. Profile page with editable personal details and the order history listing.

21. Conclusion

BOOKSTORE is a complete, well-structured full-stack web application that demonstrates the MERN stack end to end. It delivers a credible online bookstore experience for customers, including account management, full-text search with faceted filtering, a persistent cart, a simulated checkout, order history, and a review system that automatically maintains aggregate ratings. For administrators, it provides a focused dashboard for catalogue management and order fulfilment, gated by role-based access control that is enforced on the server rather than merely hidden in the interface.

Each milestone, from project setup through backend and database development to frontend integration and testing, contributes a verifiable increment that builds on the previous one. The architectural decisions, including the consistent response envelope, the defence-in-depth approach to authorisation, the derived rating fields maintained by Mongoose hooks, and the order item snapshots that insulate historical orders from later edits, reflect patterns that scale beyond a minimum viable product. The documented trade-offs, such as the simulated checkout and the non-transactional stock decrements, are clearly identified and paired with a roadmap that hardens them in well-defined stages.

The result is a platform that is straightforward to understand, easy to deploy, and ready to extend. The Phase 2 roadmap, which includes real payment integration, transactional stock management, cloud image storage, automated testing, refresh tokens, rate limiting, and email notifications, provides a clear path from the current minimum viable product to a production-grade storefront. As it stands, BOOKSTORE is a clean reference implementation of modern full-stack practice and a solid foundation for further development.